

Team Strides: Development & Database Guide

AI Team Lead

February 4, 2026

1 Prerequisites (Required Before Setup)

Ensure the following tools are installed **before** cloning the repository.

- **Node.js (LTS)** — version 18 or higher
- **npm** (included with Node.js)
- **Git**
- **VS Code** (recommended editor)

Verify installations:

```
node -v
npm -v
git --version
```

If any command fails, install the missing tool before continuing.

2 First-Time Project Setup

These steps are completed **once per machine**.

1. Clone the Repository

```
git clone https://github.com/purplegummy/Strides.git
cd Strides
```

2. Install Dependencies

```
npm install
```

3. Environment Variables

Create a file named `.env` in the project root.

Check slack for the database URL and add:

```
DATABASE_URL="..."
```

Never commit this file.

4. Generate Prisma Client

```
npx prisma generate
```

5. Verify Setup

```
npm run dev
```

Open <http://localhost:3000>. If the app loads, setup is complete.

3 Git Workflow Rules (Non-Negotiable)

- Never push directly to `main`
- Always work on a feature branch
- Never merge broken code

Standard Feature Workflow

1. Sync with main:

```
git checkout main  
git pull origin main
```

2. Create and switch to a feature branch:

```
git checkout -b feature-my-feature
```

3. Code and test locally

4. Commit changes:

```
git add .  
git commit -m "Clear description of work"
```

5. Push branch:

```
git push -u origin feature-my-feature
```

6. Open a Pull Request and wait for approval

4 Critical Concept: How Prisma Sees the World

Prisma compares your `schema.prisma` file directly to the **live remote database**. Anything missing in your local schema is treated as something to DELETE.

Git branches do NOT isolate database changes. The database is shared global state.

5 Prisma Commands Explained

`npx prisma generate`

What it does:

- Generates the Prisma Client used by the app
- Does NOT modify the database

When to run:

- After pulling new code
- After any schema change

Risk level: Safe at all times

`npx prisma db push`

What it does:

- Forces the database schema to match your local `schema.prisma`
- Deletes anything present in the database but missing locally

Key warning:

Prisma does not know about Git branches, pull requests, or unfinished features.

Risk level: High without coordination

6 The Schema Lock Protocol (MANDATORY)

The Rule

If you modify `schema.prisma` and run `db push`, you have locked the database.

Until your schema changes are shared or merged:

- No one else may safely run `npx prisma db push`
- Other developers may continue coding, but not push schema changes

What You MUST Do After a Schema Change

1. Immediately notify the team:

“I added a column to `schema.prisma`. Do not run `db push` without syncing my changes.”

2. Share your schema changes (paste or PR)
3. Merge as soon as your feature allows

Green Light: Additive Changes

Examples:

- Adding a new table
- Adding a new column

Allowed if and only if:

- The team is aware
- Everyone who may run `db push` has your schema

Red Light: Destructive Changes

Examples:

- Renaming fields
- Deleting tables or columns
- Changing column types

Protocol:

1. Notify the team
2. Ensure all work is pushed
3. Get confirmation
4. Only then run `db push`

Yellow Light: Schema Desynchronization

Scenario:

- Developer A runs `db push` with new columns
- Developer B runs `db push` without them

Result:

Prisma deletes Developer A's changes.

Correct Fix:

1. Sync schemas manually
2. Confirm schemas match
3. Then run `db push`

7 Troubleshooting

App crashes after pulling code Run:

```
npx prisma generate
```

Prisma warns about deleting data Stop immediately. You are missing schema changes.

Merge conflict in schema.prisma Manually merge both versions. Never blindly accept changes.